

to 94% of data read from the disk is wasted. Obviously, ID-consecutive vertices in a block do not imply spatial proximity (e.g., vertices of the same color are scattered on the graph topology in Fig. 2(a)). Therefore, the baseline has poor data locality.

Problem 2: Long search path. The I/O complexity of searching on a disk-based graph index is proportional to the search path length. The baseline framework uses a fixed or random vertex as the entry point for the search. However, a segment may have tens of millions of vertices, so the entry point may be far from the query. In this case, only the last few vertices in the path are near the query and likely to be in the final result. However, we need to read each vertex along the path from the disk. Fig. 2(b) shows an extreme example where the entry point is the farthest vertex from the query point. It takes five hops to search from **12** to **3**. If we start from **9**, we can reduce the hops to two. For a dataset of tens of millions of vectors, even searching for only the top-10 nearest neighbors may generate a path of hundreds of hops to achieve high accuracy. Therefore, the baseline has a long search path.

3.2 Framework Overview

We present Starling, a framework designed to enhance the I/O-efficiency of disk-based graph index for HVSS on a data segment. While a single machine may encompass multiple segments, they all share the same index layout and search strategy. Therefore, our primary focus is on one specific segment. As depicted in Fig. 2(c), Starling preserves the graph's topology while optimizing the data layout to augment the vertex utilization ratio and diminish the search path length. Moreover, it employs a block search strategy aimed at reducing disk I/Os. Below, we provide an overview of the data layout and search strategy employed by Starling:

Data layout on disk. Starling improves data locality by shuffling the data blocks in accordance with the graph topology (Fig. 2(c) left). This aligns the data layout with the search procedure, which tends to visit neighboring vertices [68]. Specifically, Starling endeavors to store a vertex and its neighbors in the same block, enabling a single disk read to fetch not only the target vertex but also other likely candidates. This approach increases the vertex utilization ratio, as each disk I/O operation brings multiple relevant vertices. Furthermore, the search procedure can more readily jump to a closer vertex from the query, potentially reducing the number of required disk I/Os.

Data layout in memory. Starling identifies query-aware entry points for the disk-based graph using an in-memory navigation graph (Fig. 2(c) right). This approach incorporates the concept of multi-layered graphs [46, 49, 60] and involves sampling a small fraction of vectors from the disk-based graph to construct the navigation graph. During this process, any in-memory graph algorithm [24, 25] can be utilized. Given a query, Starling initially explores the in-memory navigation graph to obtain query-close vertices as entry points. Subsequently, it initiates the disk-based graph search from these identified points. This methodology effectively reduces the search path length on the disk-based graph.

Search strategy. Starling uses a block search strategy to exploit data locality (Fig. 2(d)). Unlike the baseline, which explores data on a vertex basis, this strategy processes data by blocks. This way, it benefits from the optimized data layout. For each loaded block, Starling computes the distance to the query for all vertices. Then, it selects the vertices that are close to the query and checks their neighbors for new search candidates. This strategy lowers the disk operation cost by exploring more relevant data per block but increases the computation cost. We further optimize the performance with three computation-specific optimizations (§5.1).

Example 1 shows how Starling reduces disk I/Os.

EXAMPLE 1. Fig. 2(b) shows a vertex search strategy that starts from vertex **12** and reaches the result vertex **3** in five hops for a given query point. This strategy needs at least six disk I/Os to